

The Computation Graph (Optional)

The **computation graph** is a key idea in deep learning. It's how frameworks like TensorFlow visualize a complex function and efficiently compute derivatives. It breaks down a large calculation into a sequence of small, simple steps.

1. Forward Propagation (Left-to-Right) ➡

Forward propagation (or "forward prop") is the process of calculating the final output (the cost J) by moving from left to right through the graph.

Let's use a simple linear regression model as an example:

- **Model:** $a = wx + b$
- **Cost:** $J = \frac{1}{2}(a - y)^2$
- **Values:** $w = 2, b = 8, x = -2, y = 2$

We can break this down into a graph:

1. **Node 1:** $c = w \times x$
 - $c = 2 \times (-2) = -4$
2. **Node 2:** $a = c + b$
 - $a = (-4) + 8 = 4$
3. **Node 3:** $d = a - y$
 - $d = 4 - 2 = 2$
4. **Node 4 (Final Cost):** $J = \frac{1}{2}d^2$
 - $J = \frac{1}{2}(2)^2 = 2$

2. Backpropagation (Right-to-Left) ⬅

Backpropagation (or "backprop") is the process of computing the derivatives by moving backward (right-to-left) through the graph. It uses the derivative's "nudging" concept to see how each node in the graph affects the final cost J .

We want to find $\frac{\partial J}{\partial w}$ and $\frac{\partial J}{\partial b}$.

1. **Start at the end (Node 4 → Node 3):** Find $\frac{\partial J}{\partial d}$
 - $J = \frac{1}{2}d^2$. We have $d = 2$.
 - Using the calculus rule for d^2 , the derivative is just d .
 - So, $\frac{\partial J}{\partial d} = d = 2$.
 - *Intuition:* If d is nudged by ϵ , J changes by $2 \times \epsilon$.

2. **Move to Node 2:** Find $\frac{\partial J}{\partial a}$

- We know $d = a - y$. If we nudge a by ϵ , d also changes by ϵ .
- Since d changing by ϵ causes J to change by $2 \times \epsilon$ (from step 1), a nudge in a also causes J to change by $2 \times \epsilon$.
- Therefore, $\frac{\partial J}{\partial a} = 2$.

3. **Move to Node 1:** Find $\frac{\partial J}{\partial c}$ and $\frac{\partial J}{\partial b}$

- We know $a = c + b$.
- If we nudge c by ϵ , a also changes by ϵ . This causes J to change by $2 \times \epsilon$.
- Therefore, $\frac{\partial J}{\partial c} = 2$.
- If we nudge b by ϵ , a also changes by ϵ . This causes J to change by $2 \times \epsilon$.
- Therefore, $\frac{\partial J}{\partial b} = 2$. **We found one of our derivatives!**

4. **Move to the start:** Find $\frac{\partial J}{\partial w}$

- We know $c = w \times x$, and $x = -2$. So, $c = -2w$.
- If we nudge w by ϵ , c changes by $-2 \times \epsilon$.
- From step 3, we know that any change in c causes a $2 \times$ change in J .
- So, a nudge of ϵ in w causes a change of $(-2 \times \epsilon) \times 2 = -4 \times \epsilon$ in J .
- Therefore, $\frac{\partial J}{\partial w} = -4$. **We found our other derivative!**

Why Backprop is Efficient

- **Reusability:** Backprop is efficient because it **reuses calculations**. Notice that to find the derivatives for *both* w and b , we needed to know $\frac{\partial J}{\partial a}$. We only had to compute this value *once* and then reused it for both backward paths.
- In a large network, this reusability saves an enormous amount of computation, making it far more efficient than calculating each derivative independently from scratch.